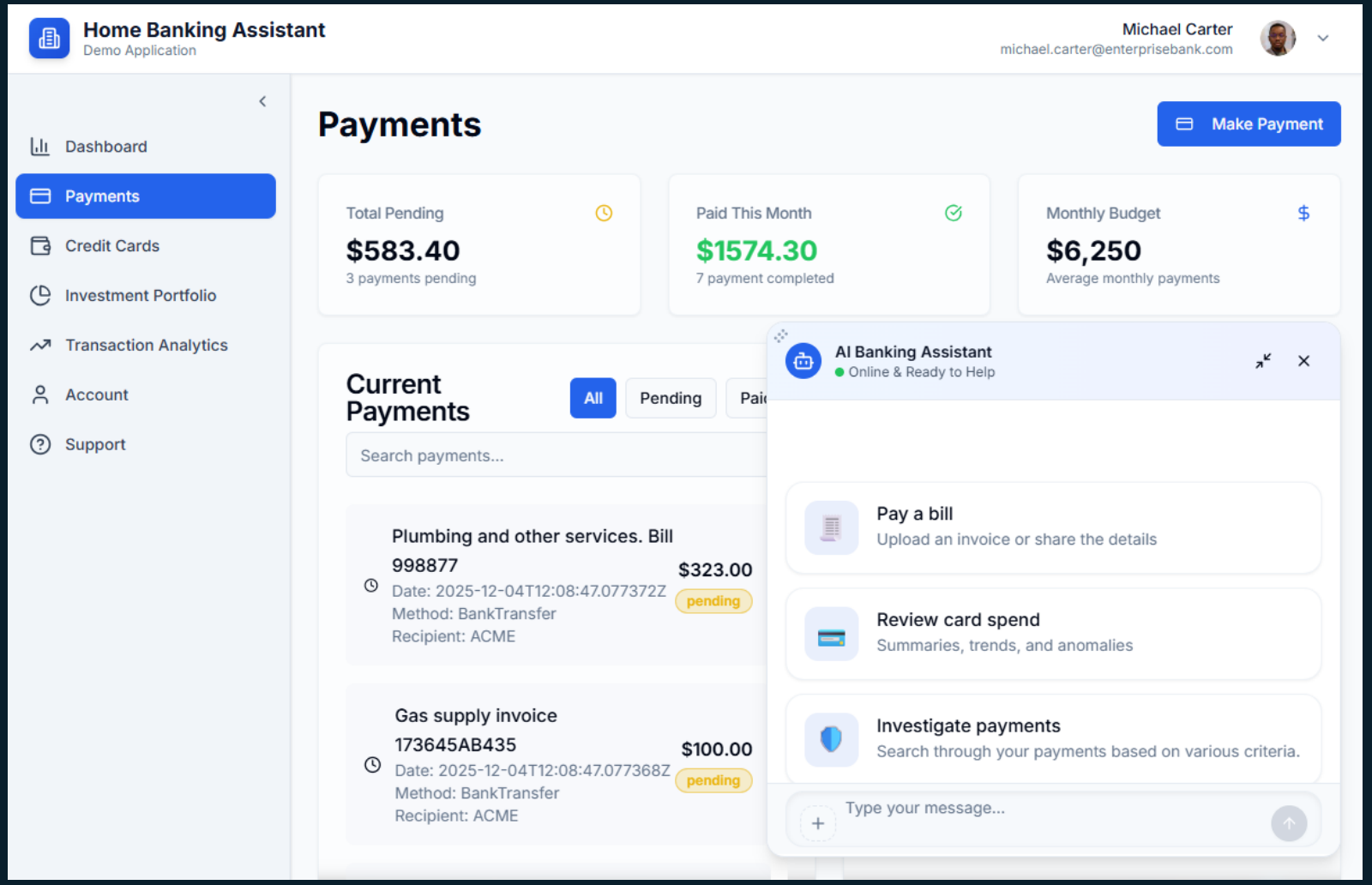
Home Banking Assistant

Conversational Agents for Banking Digital Services

https://github.com/Azure-Samples/agent-openai-python-banking-assistant

Use Case

- 📱 In-app home banking assistant
- 🏦✨ Intuitive and accessible banking services
- 🔗 Connect and reuse existing banking services
- 🧾 OCR-driven payments
- 🧩 Reusable multi-agent architecture



Business Benefits

Enhance Home Banking User Experience With More Intuitive and Accessible Services

Frictionless Conversational Home Banking

Personalized Financial Guidance

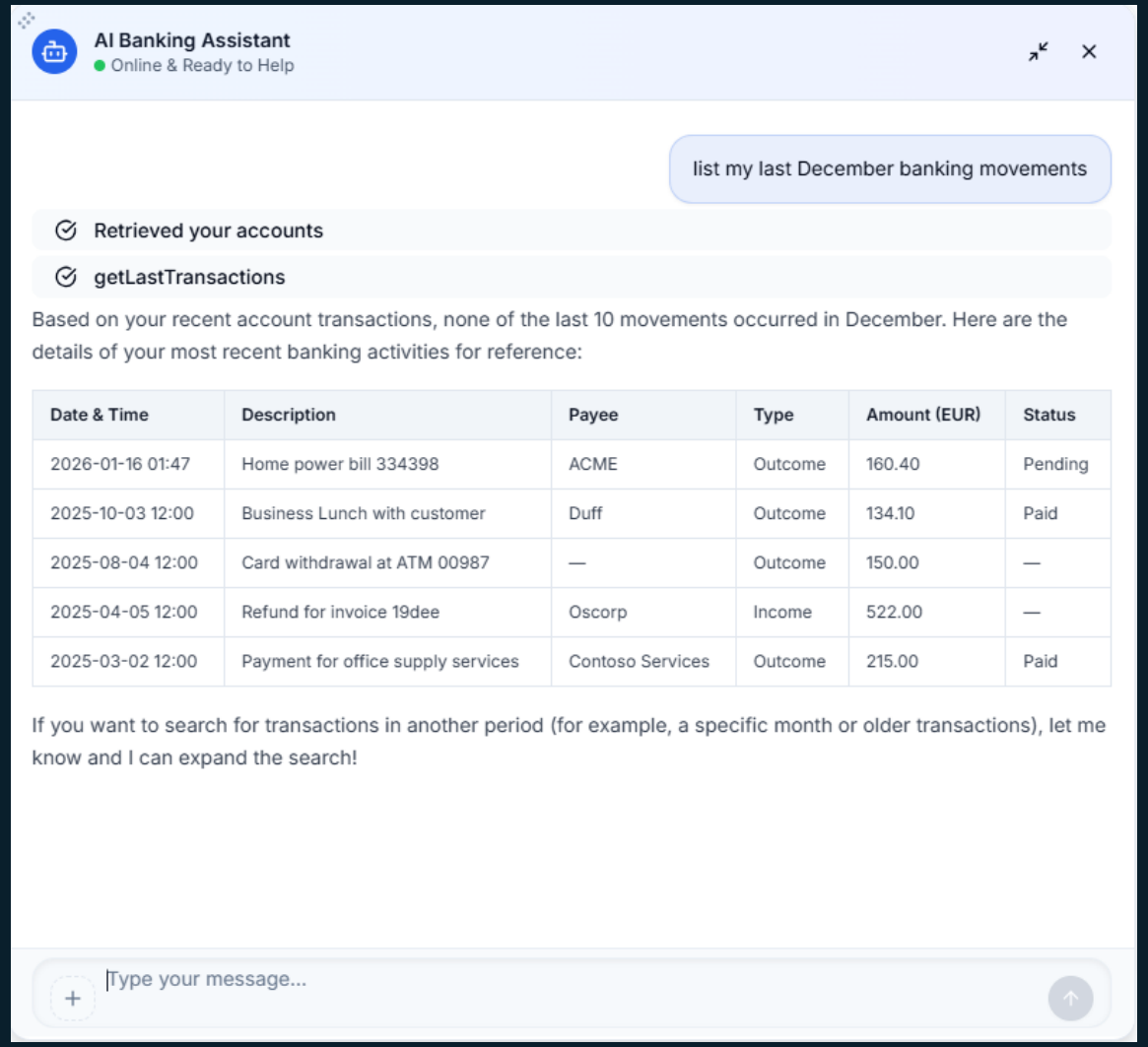
Accessibility and Inclusivity

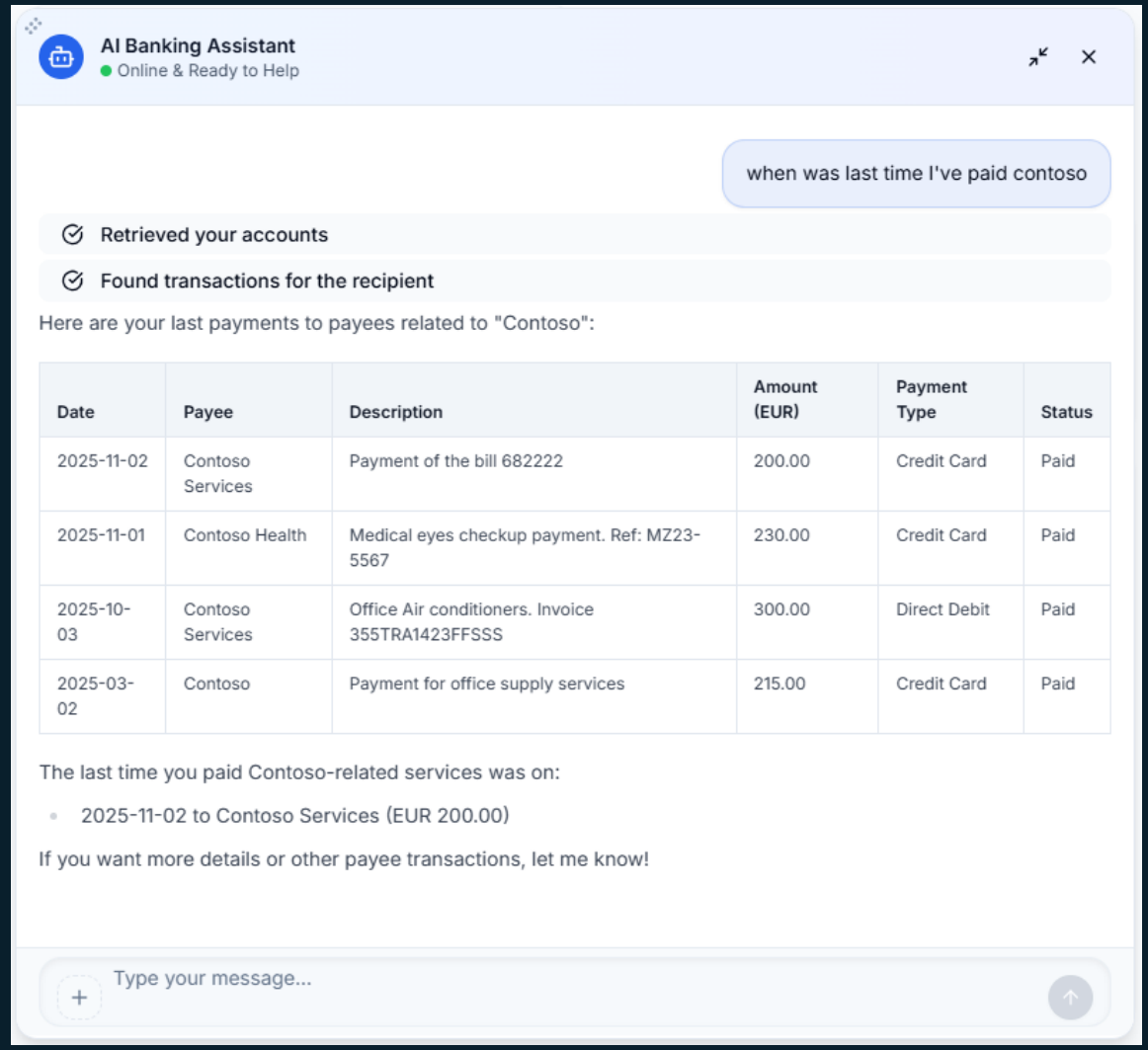
24/7 Assisted Service

Customer Engagement and Retention

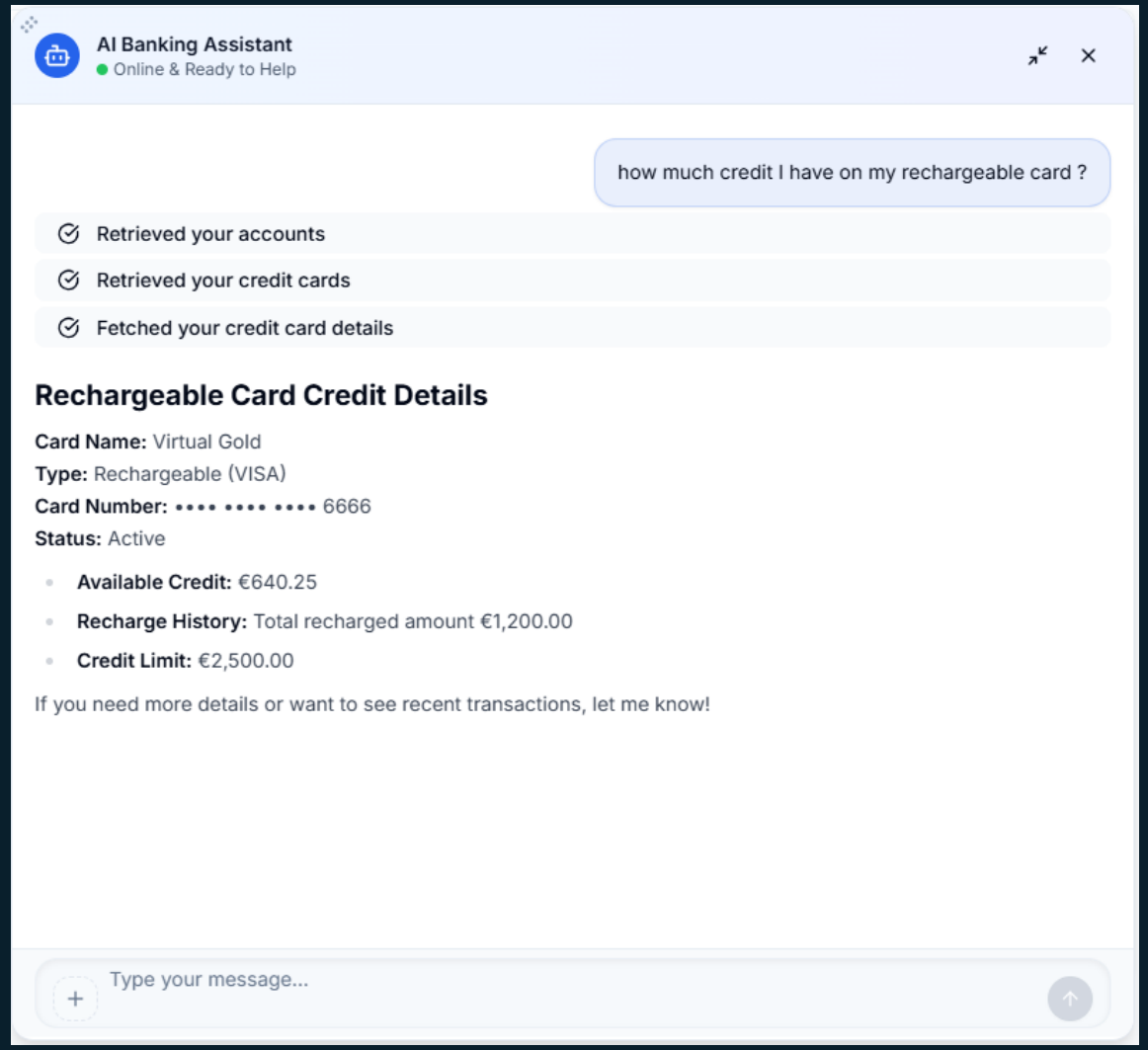
Azure AI Foundry and Microsoft Agent Framework

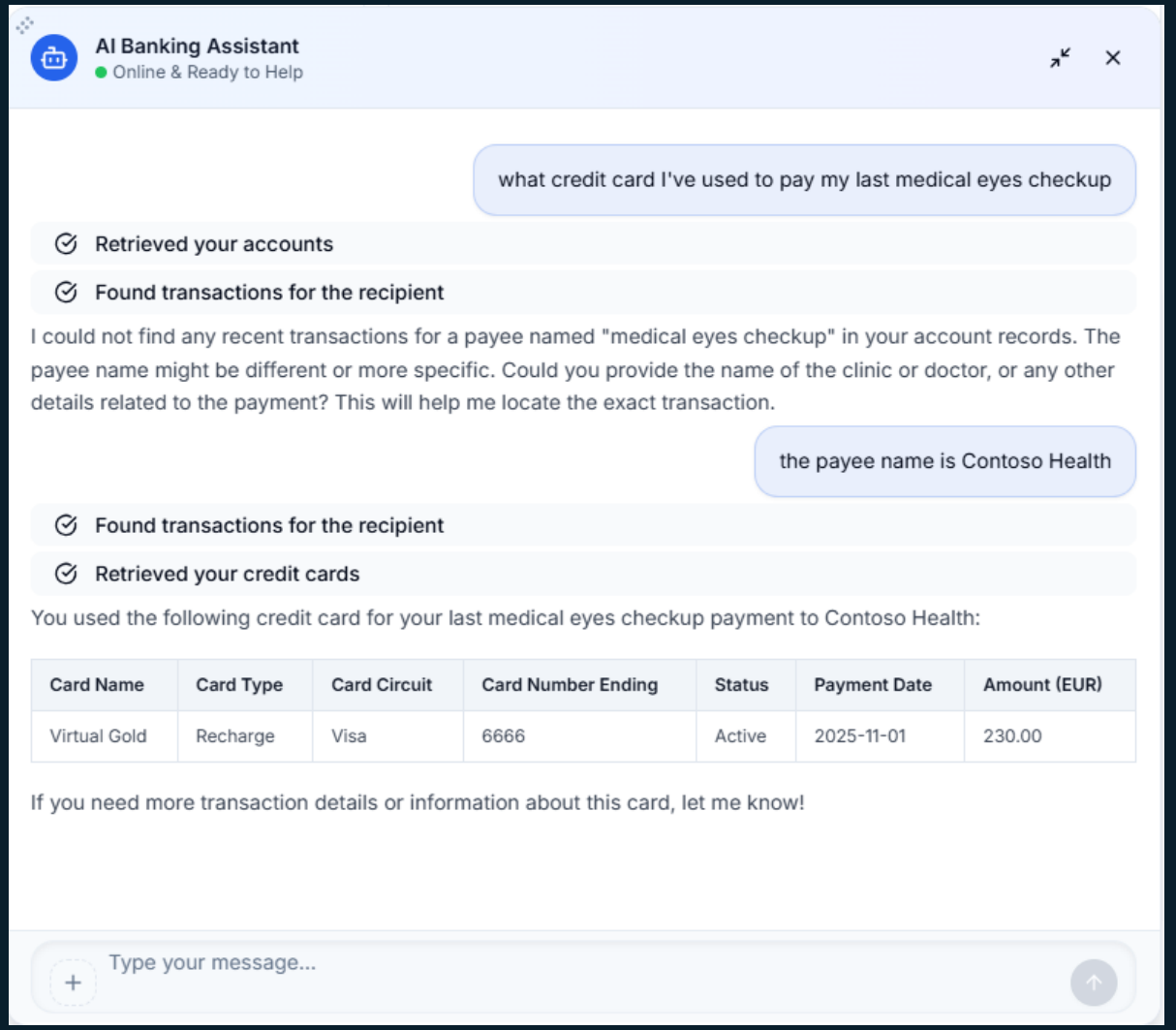
Key Features – Banking Movements History





Key Features – Review Account and Credit Cards

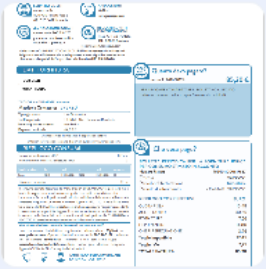




Key Features – Payment Submission with OCR

AI Banking Assistant
Online & Ready to Help

please help me to pay this bill



- Data extracted from the uploaded image
- Retrieved your accounts
- Found transactions for the recipient
- Fetched your account details

Here is the information extracted from your invoice:

Field	Value
Payee Name	GORI
Invoice/Bill ID	9524011000817857
Invoice Date	2024-05-08
Total Amount	€85,20
Address	Via Trentola, 211 80056 Ercolano (Na)

Type your message...

AI Banking Assistant
Online & Ready to Help

Total Amount	€85,20
Address	Via Trentola, 211 80056 Ercolano (Na)

The bill appears unpaid for payee "GORI".
Available payment methods on your account:

- Bank Transfer
- Visa (Primary Platinum)
- Visa (Secondary Gold)

Please confirm that the extracted invoice details are correct and let me know your preferred payment method (Bank Transfer, Primary Platinum Visa, or Secondary Gold Visa) so I can proceed.

use my rechargeable one

- Retrieved your credit cards
- Fetched your credit card details

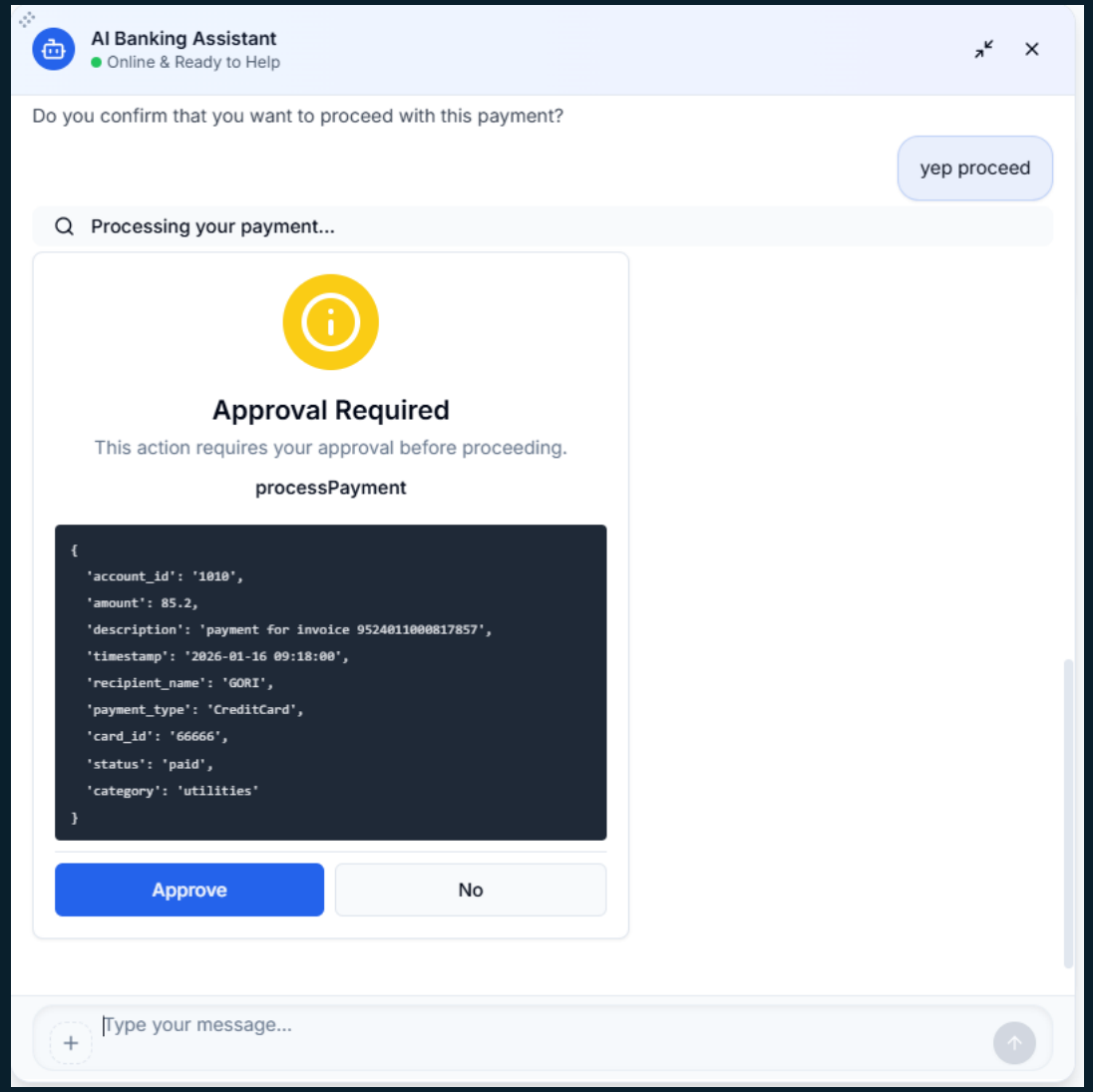
Here are the payment details for confirmation:

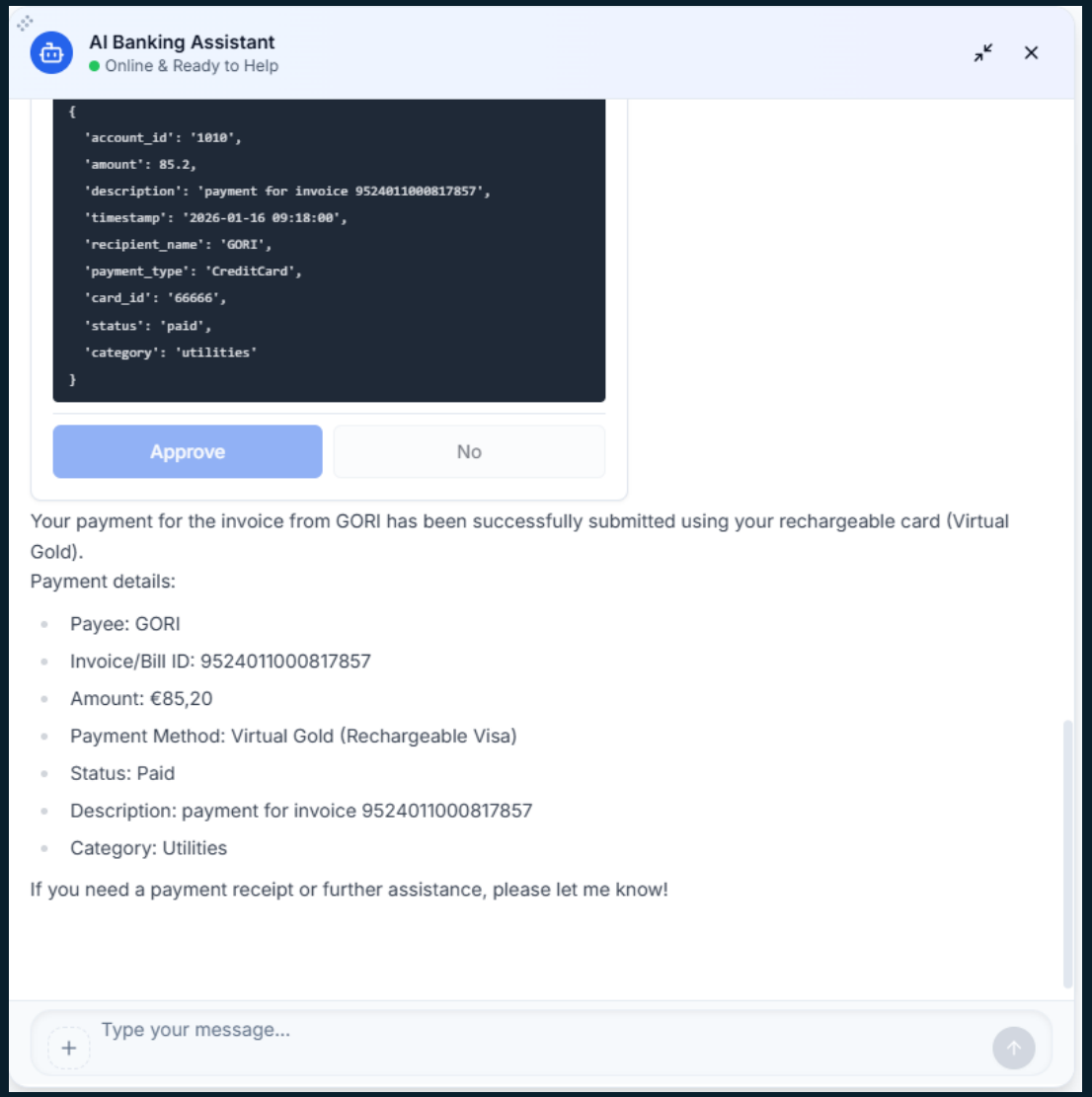
- Payee Name: GORI
- Invoice/Bill ID: 9524011000817857
- Amount: €85,20
- Payment Method: Virtual Gold (Rechargeable Visa)
- Card Balance: €640,25

Payment Description: payment for invoice 9524011000817857
Category: Utilities
Do you confirm that you want to proceed with this payment?

Type your message...

Key Features – Payment User Confirmation

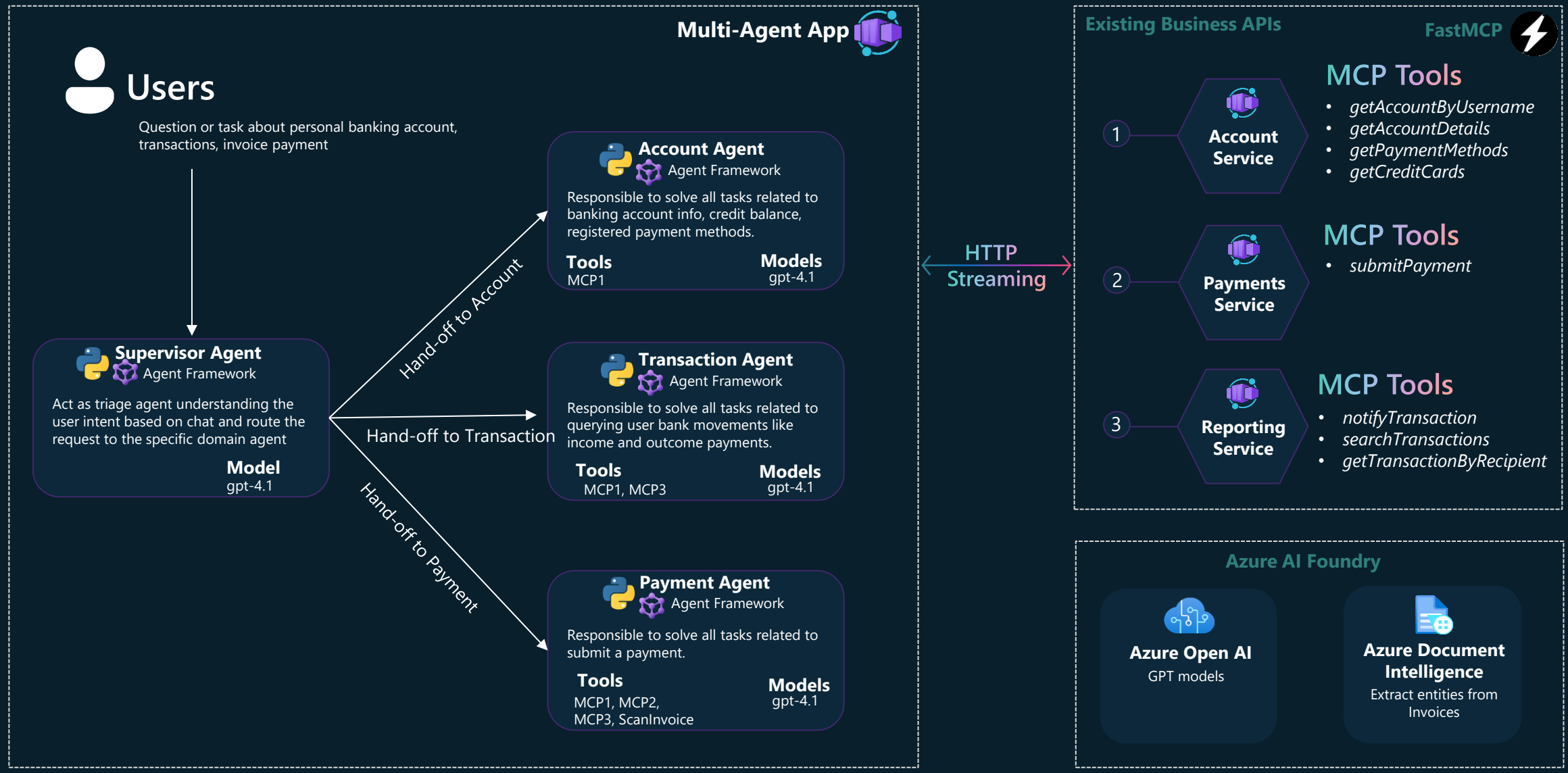




Solution Architecture

Microsoft has different ways of building agents

Agents and Banking API Architecture



Recommended "Golden Path" Architecture

3P agents

BYO resources

- **Cosmos DB** Thread storage
- **Key vault** Connections
- **Azure Storage** File storage
- **Azure AI Search** File search index
- **Azure Managed Redis** Agent Memory

Microsoft Foundry

Standard setup

Observability

Built-in AI tools

File search

Code interpreter

Agents

Models

One-click publishing or M365 Agents SDK

AI tool resources

- Azure AI Search
- Azure Functions
- Logic Apps
- Azure Managed Redis
- Grounding with Bing Search

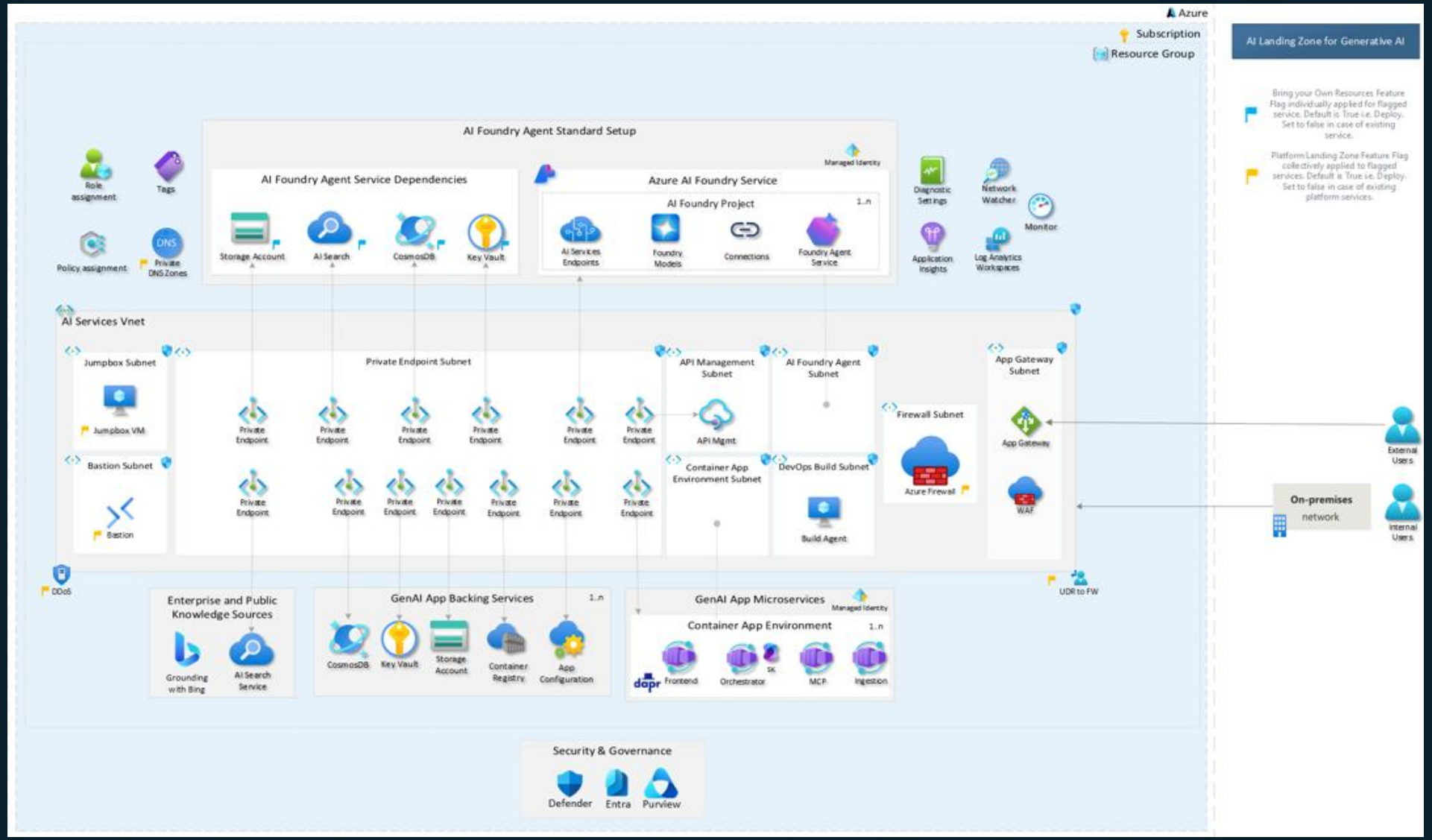
Microsoft Agent Framework

Multi-agent orchestrator

Azure Containers or Hosted agents in Foundry Agent Service

- Fabric
- SharePoint
- External APIs (OpenAPI specs)
- MCP servers
- A2A Servers

Deploy with AI Landing Zone



https://github.com/Azure/AI-Landing-Zones

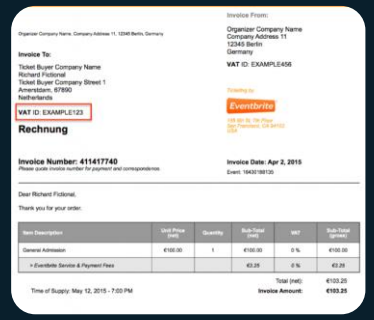
Technical Architecture

Agentic Building Blocks

Single Agent - ReAct Planning with Tools Calling

Users

Pay this bill for me



bill-abc123.jpeg

Bill abc123 successfully paid

Home Banking Agent

Tools:

- scanImage (filename)
- transactionHistory (billId)
- paymentService (billId, Amount, Payee)

Instructions:

- You are a home banking assistant allowing users to pay the bill uploading a picture
- Always check if a bill has already been paid before submitting a payment
- Confirm the payment result

Planning

while (new tools execution request)

1. **scanImage** (bill-abc123.jpeg) → abc123, 100$, payeename
2. **transactionHistory** (abc123) → no results
3. **paymentService** (abc123, 100$,payeename) → ok

Single Agent Scaling Problem

As the system grows you might run into scaling challenges

Too many tools. Tool hallucinations

Agent context (a.k.a. prompt) grows too much and it fails to follow instructions

Handling complex and dynamics tasks spanning different business domains

Multi agent architecture opportunities

Manageability – Modular agents reduce development and testing complexity

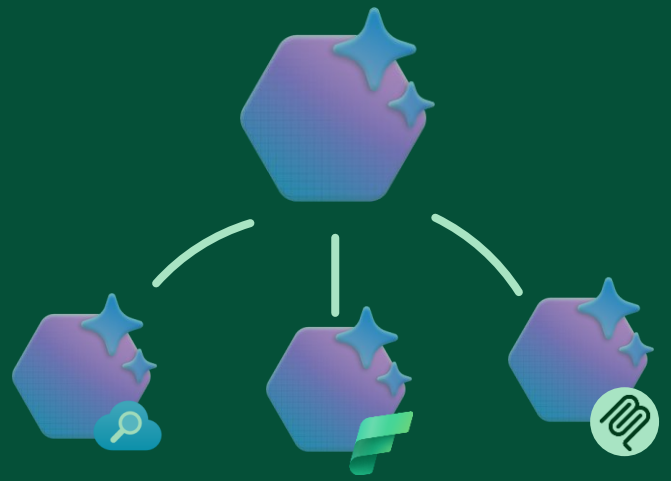
Predictability – More control over application flow using structured agents communication

Flexibility – Ease to incorporate new agents as solution domains increase

Multi-Agent Systems Orchestration

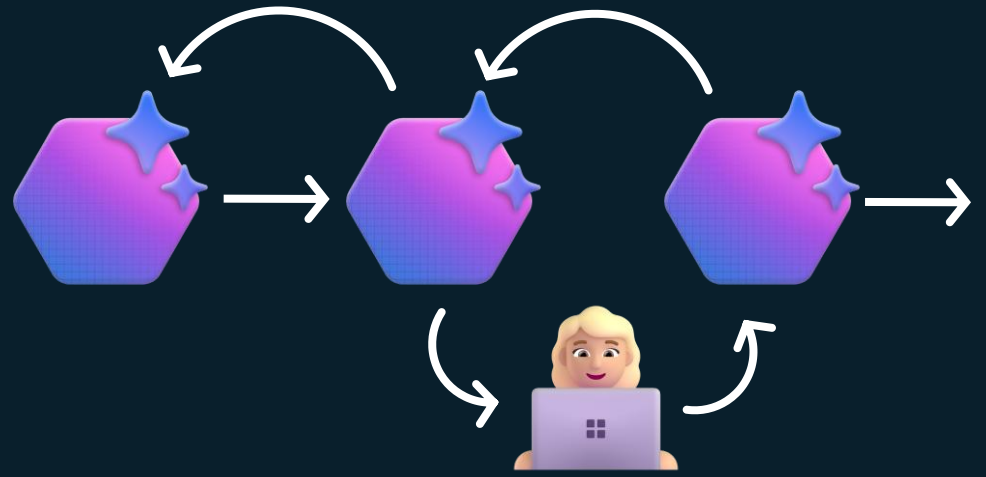
Home Banking Agents Orchestration Approach

Agent Orchestration



LLMs dynamically decide the control flow and direct their own processes and tool usage, maintaining control over how they accomplish tasks.

Workflow Orchestration

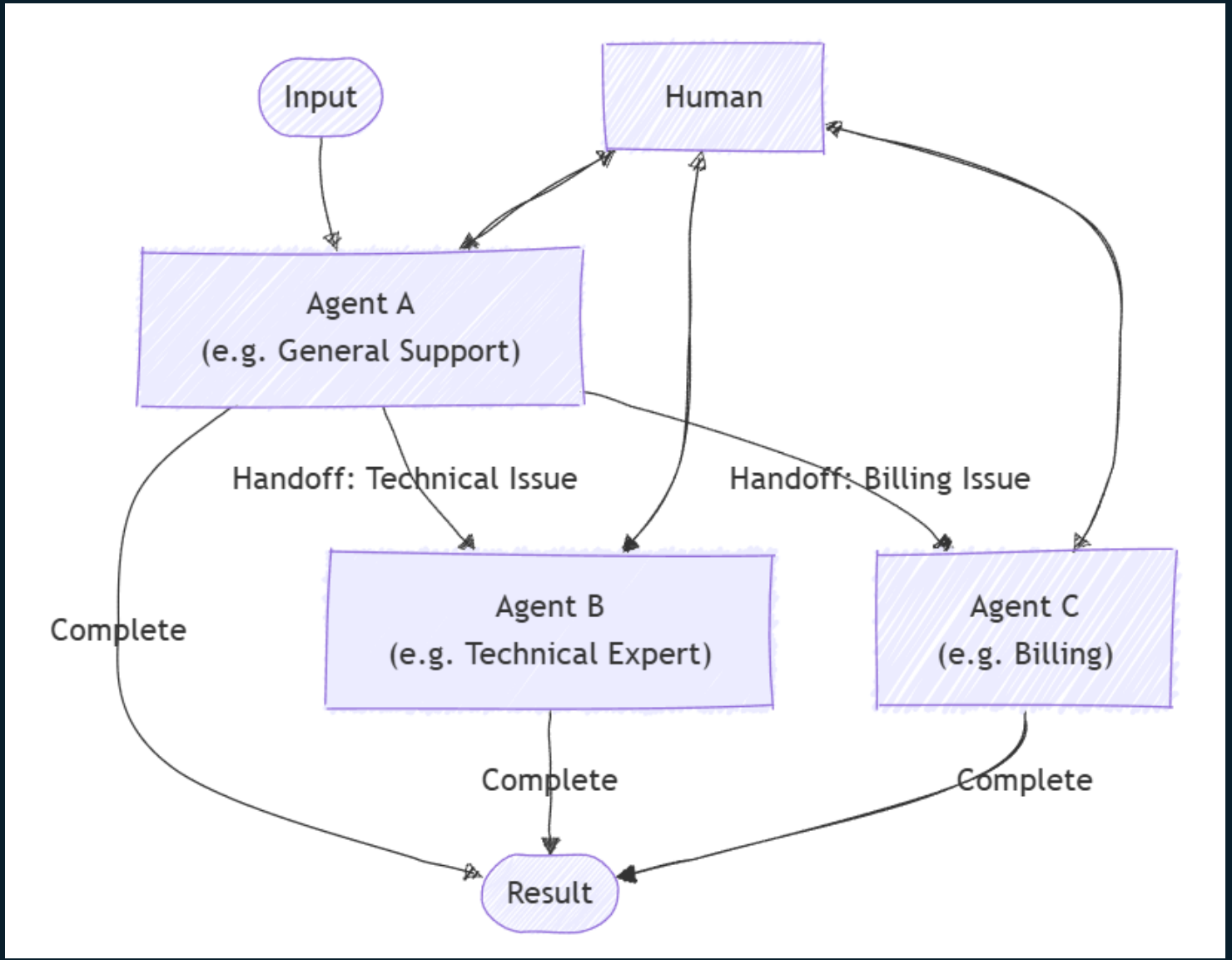


You decide the control flow orchestrating LLMs, Agents, tools and more via predefined code paths.

Handoff pattern

The handoff orchestration pattern enables dynamic delegation of tasks between specialized agents. Each agent can assess the task at hand and decide whether to handle it directly or transfer it to a more appropriate agent based on the context and requirements.

This pattern addresses the home banking assistant scenario as the optimal agent for a task isn't known upfront

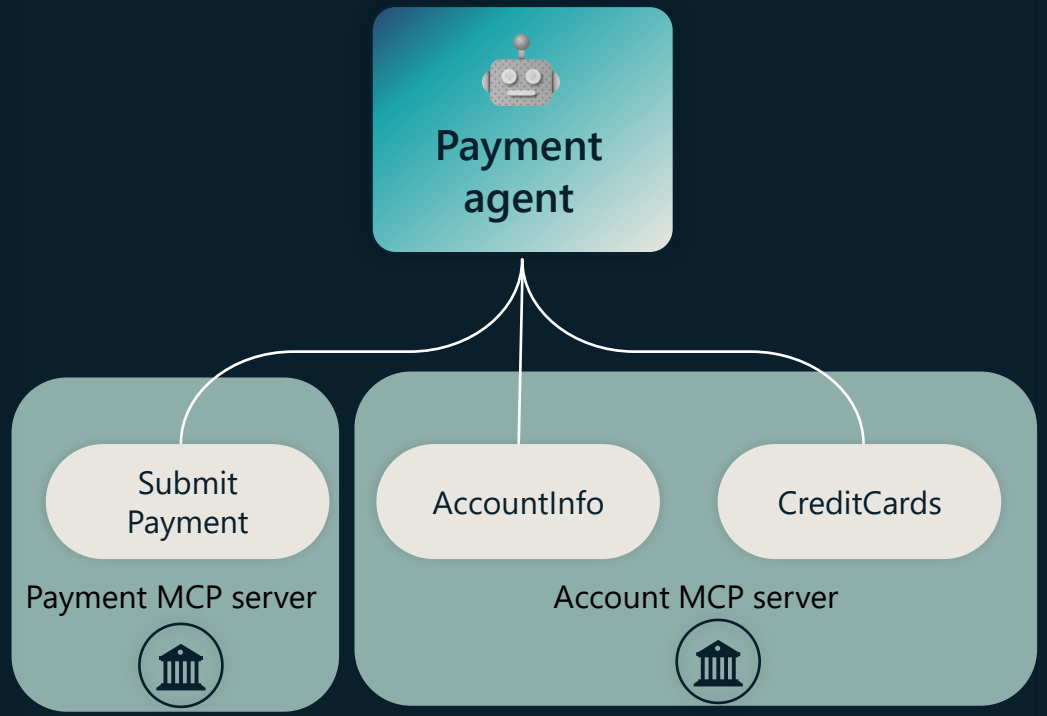


Agentic Pattern - Tools Calling with MCP

Simplify integration with AI apps and agents using Model Context Protocol

MCP is a standardized way to connect agents to your data and api. It's like a USB-C port for your agents.

Connect your agents to MCP-enabled connectors, unlocking the latest actions and knowledge made available.

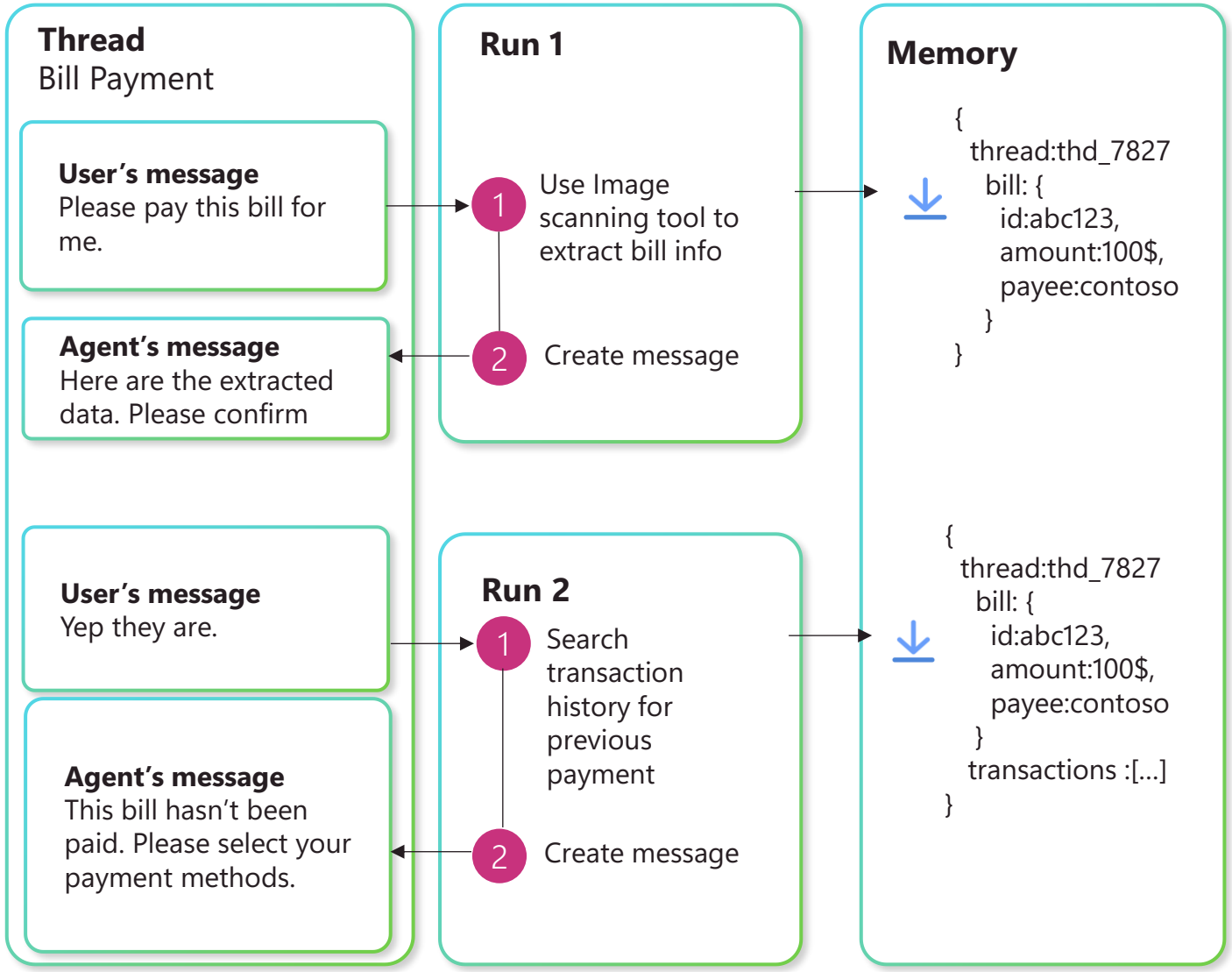


Learn more at **aka.ms/mcs-mcp**

Memory & Conversation State

The AgentThread and Workflow Checkpoint abstraction retain conversation history across turns and sessions, ensuring agents have relevant context for long-running dialogues.

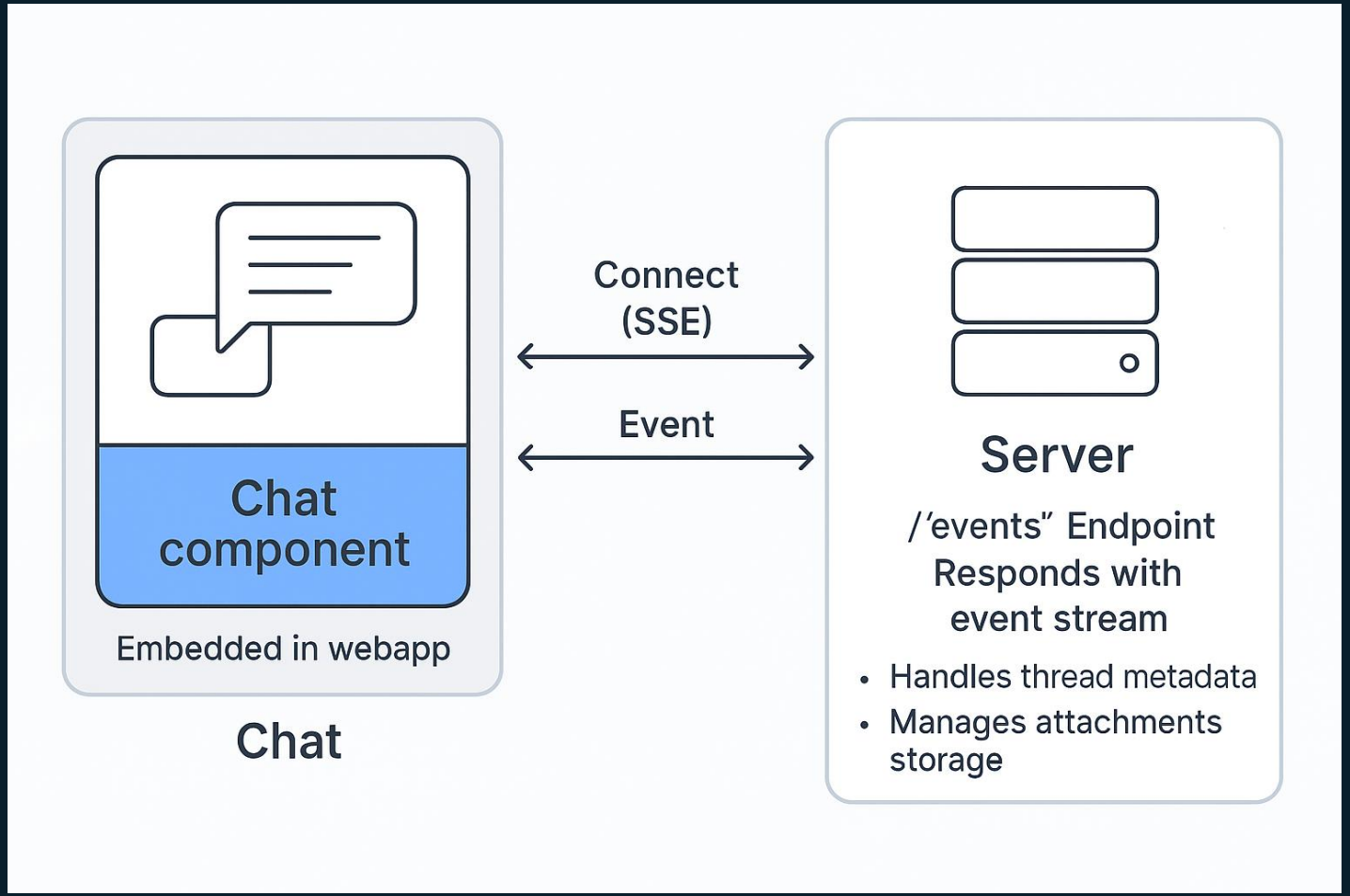
Agent Framework provides **short-term memory** (session-scoped, immediate context) and **long-term memory** (persistent data for user preferences, facts, etc.) through integrations with memory, no-sql, vector DBs.

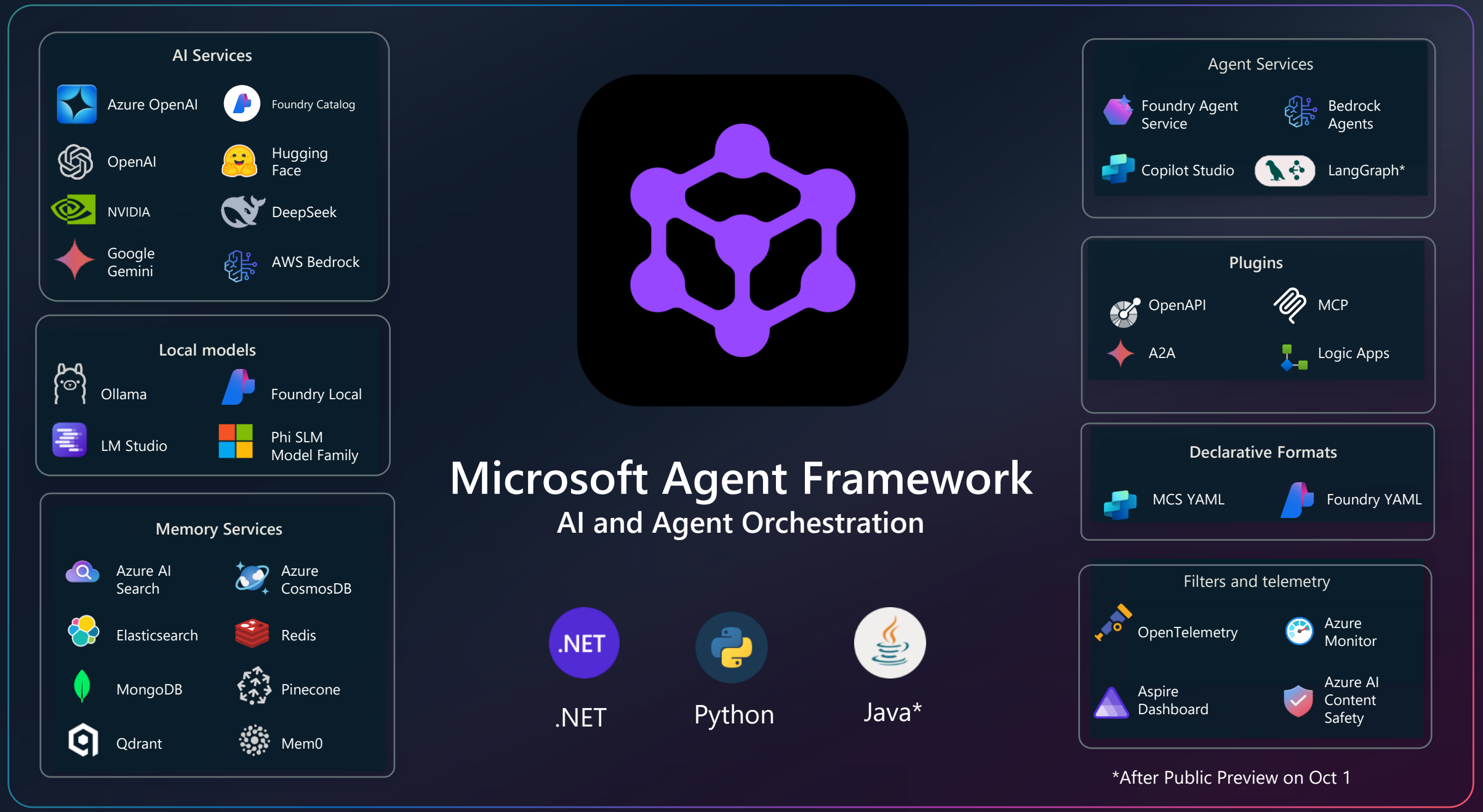


UI-to-Agent Protocol

Standardize Chat-To-Agent events communication with OpenAI Chatkit inspired protocol.

- Structured **event streaming** (thread.created, thread.item.done, etc.)
- Real-time agent **progress notifications** during task execution
- **Tool approval** widget for user confirmation before sensitive operations execution
- Support for multi-modal attachments
- **Client-managed widgets**: Pre-built React components are registered on the client and referenced by name from the server
- Chat component is **reusable** and can be embedded into any web applications





Agent Framework Key Features

Agentic Core Patterns - Chat Clients, Agent Abstractions, Tools, MCP, Threads
Middlewares - Pre/Post hook for LLM calls, tool executions.
Context Providers - Advanced memory pattern: dynamic extension, semantic search with Redis.
Dev UI - Interactive developer UI for agent development, testing, and debugging
Human-in-the-loop - Use tools with user approvals.
Multi-Agent Workflows - Connect agents and deterministic functions using data flows with streaming.
Observability – Multi-agent logging, tracing and evaluation
Multi-Agent LLM Orchestration Patterns - Sequential, Loop, Concurrent+Aggregator, Magentic, Hand-Off, Group Chat
A2A – Agent 2 Agent protocol support.

https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview

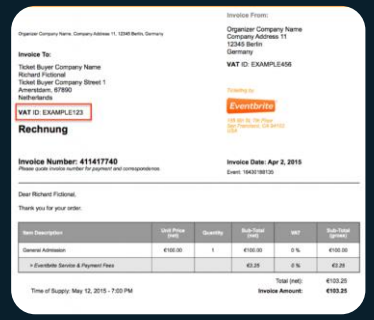
Project Setup and Local Development

1. *Provision azure resources and deploy the solution*
   - *Review and follow quick deploy instructions*
   - *azd up will automatically provision Azure resources and deploy the solution on ACA.*
2. *Start local development and debug*
   - *Review and follow local dev and debug instructions (copilot ready)*
   - *Review project repo structure and technology stack info (copilot ready)*
   - *Run agents, frontend and mocked business services locally*
   - *Use VS code for code debugging (launch.json pre-built for you)*
   - *Check VS code console for app logging*
   - *Check AppInsights-Performance blade for Agents tracing*
3. *Test banking web app*
   - *Azure: Point your browser at ACA app ca-web-xxxx url (see azd up log)*
   - *Local: Point your browser at http://localhost:5170*
4. *Start customization*
   - *Review customization guidance*
   - *Add/remove agents*
   - *Change hand-off orchestration logic*
   - *Add/remove custom tools and mcp servers*
   - *Add support for different/custom chat protocol*

Single Agent - ReAct Planning with Tools Calling

Users

Pay this bill for me



bill-abc123.jpeg

Bill abc123 successfully paid

Home Banking Agent

Tools:

- scanImage (filename)
- transactionHistory (billId)
- paymentService (billId, Amount, Payee)

Instructions:

- You are a home banking assistant allowing users to pay the bill uploading a picture
- Always check if a bill has already been paid before submitting a payment
- Confirm the payment result

Planning

while (new tools execution request)

1. **scanImage** (bill-abc123.jpeg) → abc123, 100$, payeename
2. **transactionHistory** (abc123) → no results
3. **paymentService** (abc123, 100$,payeename) → ok

Define an Azure OpenAI Agent with tools

Define a chat agent

```
return Agent(
client=self.azure_chat_client,
instructions=full_instruction,
name=PaymentAgent.name,
tools=[account_mcp_server,
       transaction_mcp_server,
       payment_mcp_server,
      self.document_scanner_helper.scan_invoice])
```

code

Define a tool

```
@tool
def scan_invoice(
    self,
    blob_name: Annotated[str, Field(description="the path to the file containing the imag
) -> Annotated[str, Field(description="Returns a JSON string containing extracted invoice
    """function to scan invoice and bill documents and extract relevant fields."""
    try:
        scan_result = self.scan(blob_name)
        logger.debug("Scan result: %s", scan_result)
        # Convert dictionary to JSON string
        return json.dumps(scan_result, indent=2)
    except Exception as e:
        logger.error("Error scanning invoice with blob name [%s]: %s", blob_name, str(e))
        return json.dumps({"error": f"Failed to scan invoice: {str(e)}"})
```

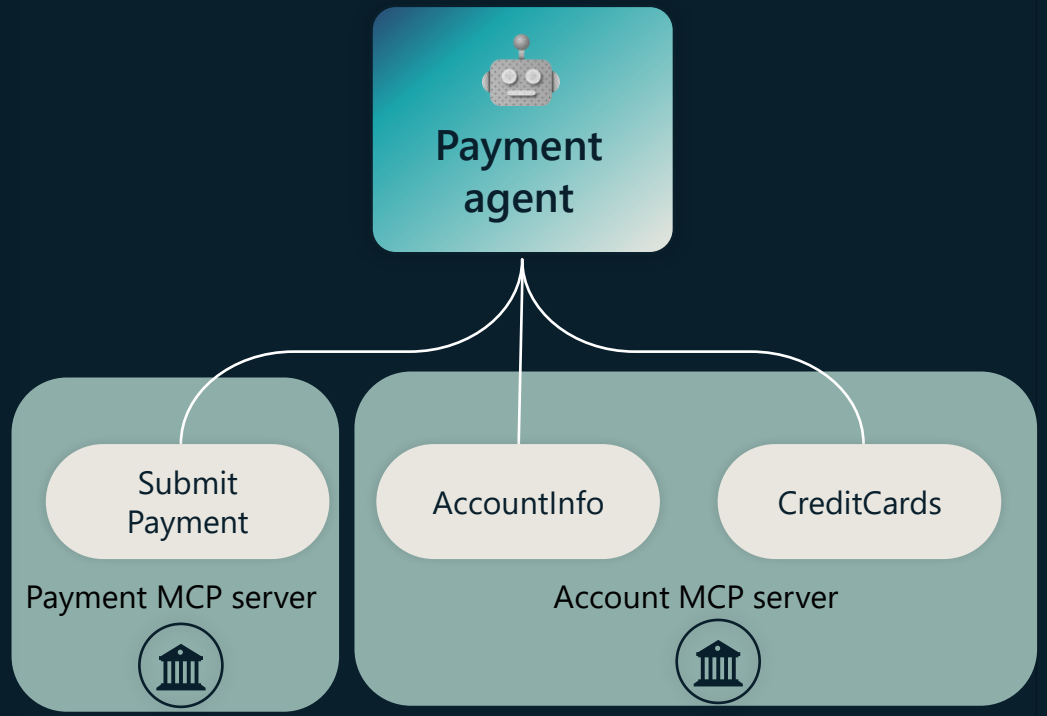
code

Agentic Pattern - Tools Calling with MCP

Simplify integration with AI apps and agents using Model Context Protocol

MCP is a standardized way to connect agents to your data and api. It's like a USB-C port for your agents.

Connect your agents to MCP-enabled connectors, unlocking the latest actions and knowledge made available.



Learn more at **aka.ms/mcs-mcp**

HTTP Streaming MCP Tools

Define an MCP endpoint for your service

```python
from fastmcp import FastMCP
from services import AccountService, UserService
from typing import Annotated

user_service = UserService()
account_service = AccountService()
mcp = FastMCP("Account MCP Server")

@mcp.tool(name="getAccountsByUserName", description="Get the list of all accounts for a specific user")
def get_accounts_by_user_name(userName: Annotated[str, "username of logged user"]):
    return user_service.get_accounts_by_user_name(userName)

@mcp.tool(name="getAccountDetails", description="Get account details and available payment methods")
def get_account_details(accountId: Annotated[str, "unique identifier for the user account"]):
    return account_service.get_account_details(accountId)
```

code

Define an MCP client: tools for your agent.

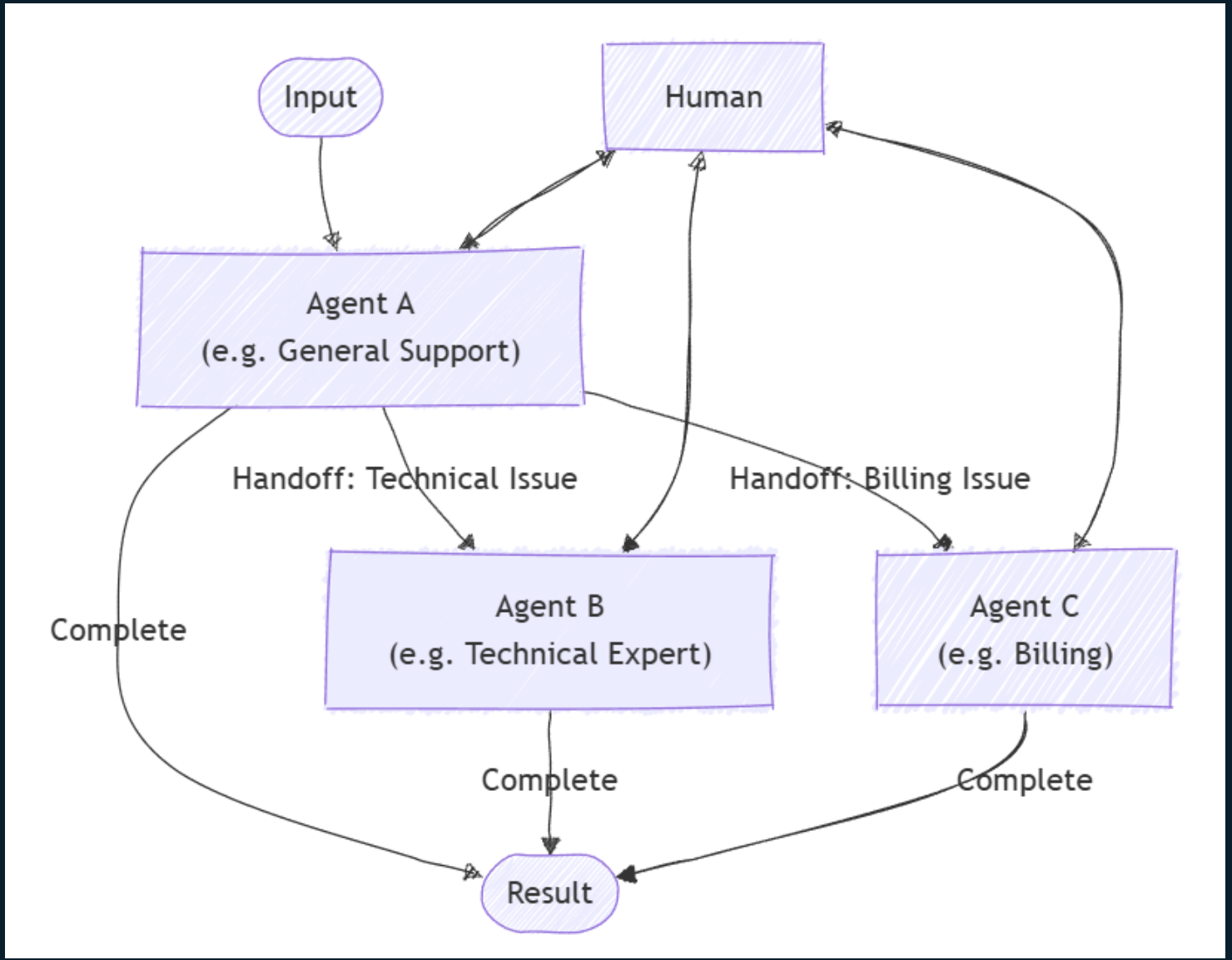
```python
logger.info("Initializing Transaction MCP server tools ")
transaction_mcp_server = MCPStreamableHTTPTool(
  name="Transaction MCP server client",
  url="https://accounts-mcp-server:8080/mcp"
)
await transaction_mcp_server.connect()
```

code

Handoff pattern

The handoff orchestration pattern enables dynamic delegation of tasks between specialized agents. Each agent can assess the task at hand and decide whether to handle it directly or transfer it to a more appropriate agent based on the context and requirements.

This pattern addresses the home banking assistant scenario as the optimal agent for a task isn't known upfront



Multi-Agent Orchestration – Hand-Off pattern

MAF hand-off orchestrator:
HandoffBuilder

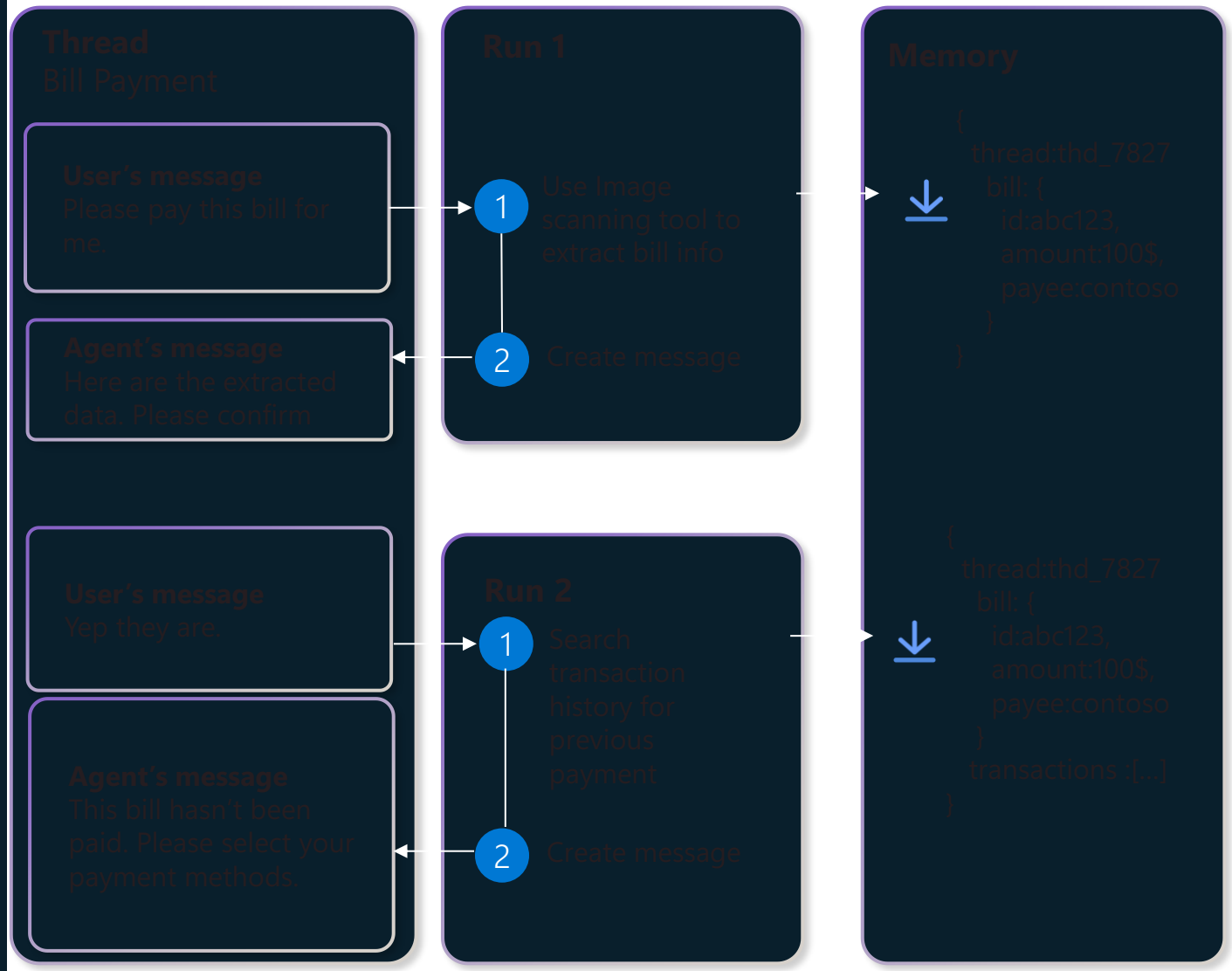
```
self.workflow = (
    HandoffBuilder(
        name="banking_assistant_handoff",
        participants=[triage_agent,
                      await self.account_agent.build_af_agent(),
                      await self.transaction_agent.build_af_agent(),
                      await self.payment_agent.build_af_agent()],
    )
    .set_coordinator("triage_agent")
    .with_termination_condition(
        # Terminate after 20 user messages
        # Count only USER role messages to avoid counting agent responses
        lambda conv: sum(1 for msg in conv if msg.role.value == "user") >= 20
    )
    .with_checkpointing(HandoffOrchestrator.checkpoint_storage)
    .build()
)
```

code

Memory & Conversation State

The AgentThread and Workflow Checkpoint abstraction retain conversation history across turns and sessions, ensuring agents have relevant context for long-running dialogues.

Agent Framework provides **short-term memory** (session-scoped, immediate context) and **long-term memory** (persistent data for user preferences, facts, etc.) through integrations with memory, no-sql, vector DBs.



Memory – Long Running Conversations

Retrieve last checkpoint per thread_id

```python
# try to retrieve checkpoint for the given thread_id. If None, we start a new conversation.
checkpoint = await checkpoint_storage.get_latest(workflow_name=self.workflow.name) # type: ignore
workflow_id = self.workflow.id  # type: ignore

if checkpoint is None:
    # Start a new conversation. This is the first user message.
    async for event in self.workflow.run(user_message, stream=True):# type: ignore
        yield event # type: ignore
else:
    #Resuming an existing conversation.
    async for event in await self._resume_workflow_with_response(checkpoint_storage,
                                                                  checkpoint.checkpoint_id,
                                                                  user_message):
        yield event
```

code

Resume the hand-off workflow with checkpoint id

```python
events = self.workflow.run(checkpoint_id=checkpoint_id,    #type: ignore
                           checkpoint_storage=checkpoint_storage,
                           stream=True)

responses: dict[str, object] = {}

#We need to collect all workflow events otherwise we get concurrent workflow execution error when trying to resume.
consumed_events = [event async for event in events]
for event in consumed_events:
    if event.type == "request_info":
        if isinstance(event.data, HandoffAgentUserRequest):
            responses[event.request_id] = HandoffAgentUserRequest.create_response(user_message)
            return self.workflow.run(responses=responses, checkpoint_id=checkpoint_id, #type: ignore
                                     checkpoint_storage=checkpoint_storage,
                                     stream=True)
        else:
            raise AgentFrameworkException(f"RequestInfoEvent [{event.request_id}] found in the checkpoint [{checkpo
```

code

HITL – Tool Approval

Define a tool approval for an mcp tool

```
logger.info("Initializing Payment  MCP server tools ")
payment_mcp_server = MCPStreamableHTTPTool(
  name="Payment MCP server client",
  url=self.payment_mcp_server_url,
  approval_mode = { "always_require_approval": ["processPayment"] })
```

code

Resume a conversation (workflow checkpoint) with tool execution approval

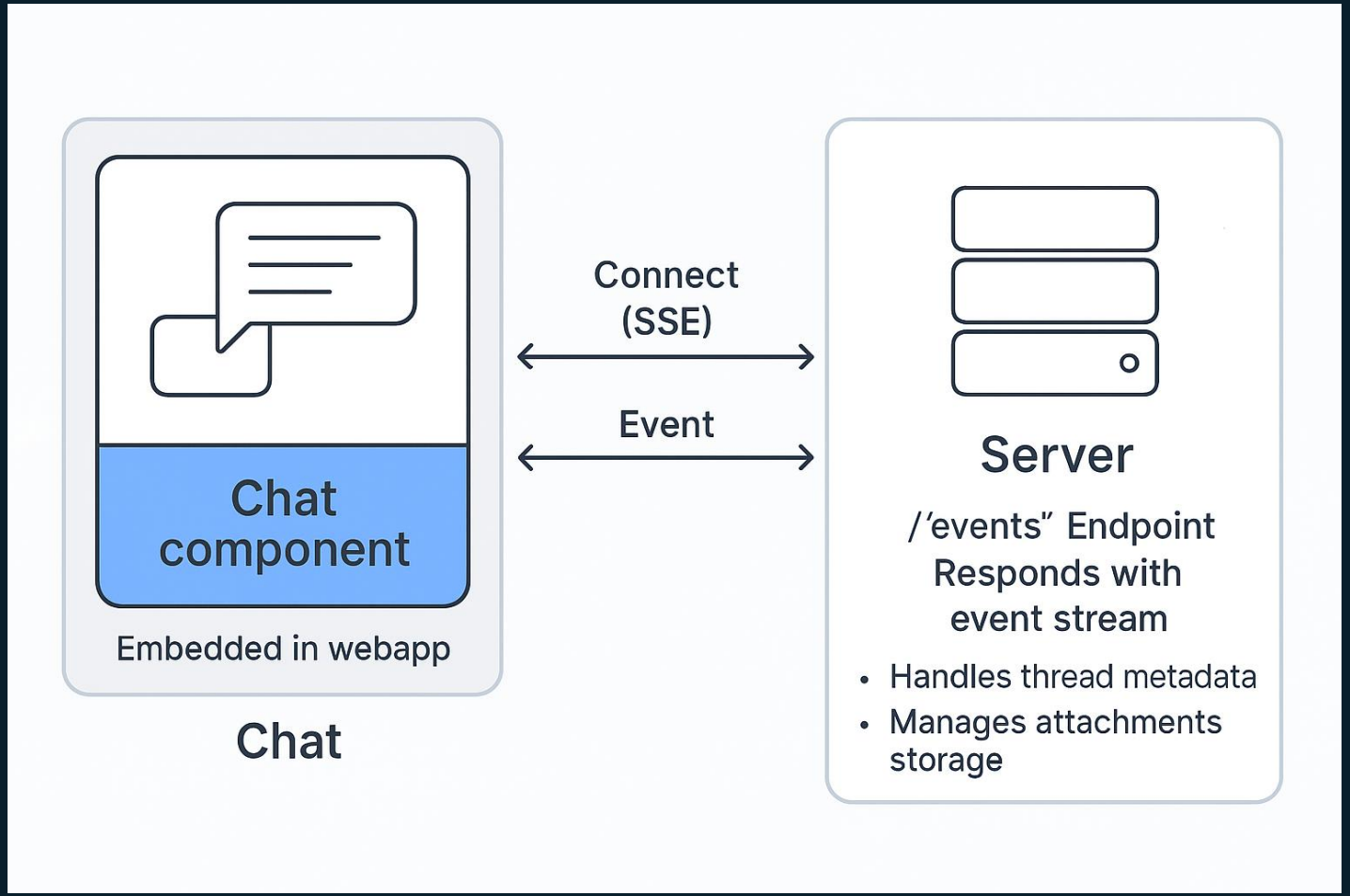
```
consumed_events = [event async for event in events]
for event in consumed_events:
    yield event
    if event.type == "request_info":
        if isinstance(event.data, Content) and event.data.type == "function_approval_request":
                responses[event.request_id] = event.data.to_function_approval_response(approved=approved)
                async for event in self.workflow.run(responses=responses, #type: ignore
                                                     checkpoint_id=checkpoint.checkpoint_id,
                                                     checkpoint_storage=checkpoint_storage, stream=True)
                    yield event
        else:
            raise AgentFrameworkException(f"RequestInfoEvent [{event.request_id}] found in the checkpoint
```

code

UI-to-Agent Protocol

Standardize Chat-To-Agent events communication with OpenAI Chatkit inspired protocol.

- Structured **event streaming** (thread.created, thread.item.done, etc.)
- Real-time agent **progress notifications** during task execution
- **Tool approval** widget for user confirmation before sensitive operations execution
- Support for multi-modal attachments
- **Client-managed widgets**: Pre-built React components are registered on the client and referenced by name from the server
- Chat component is **reusable** and can be embedded into any web applications



UI – Streaming agents response, progress, widgets

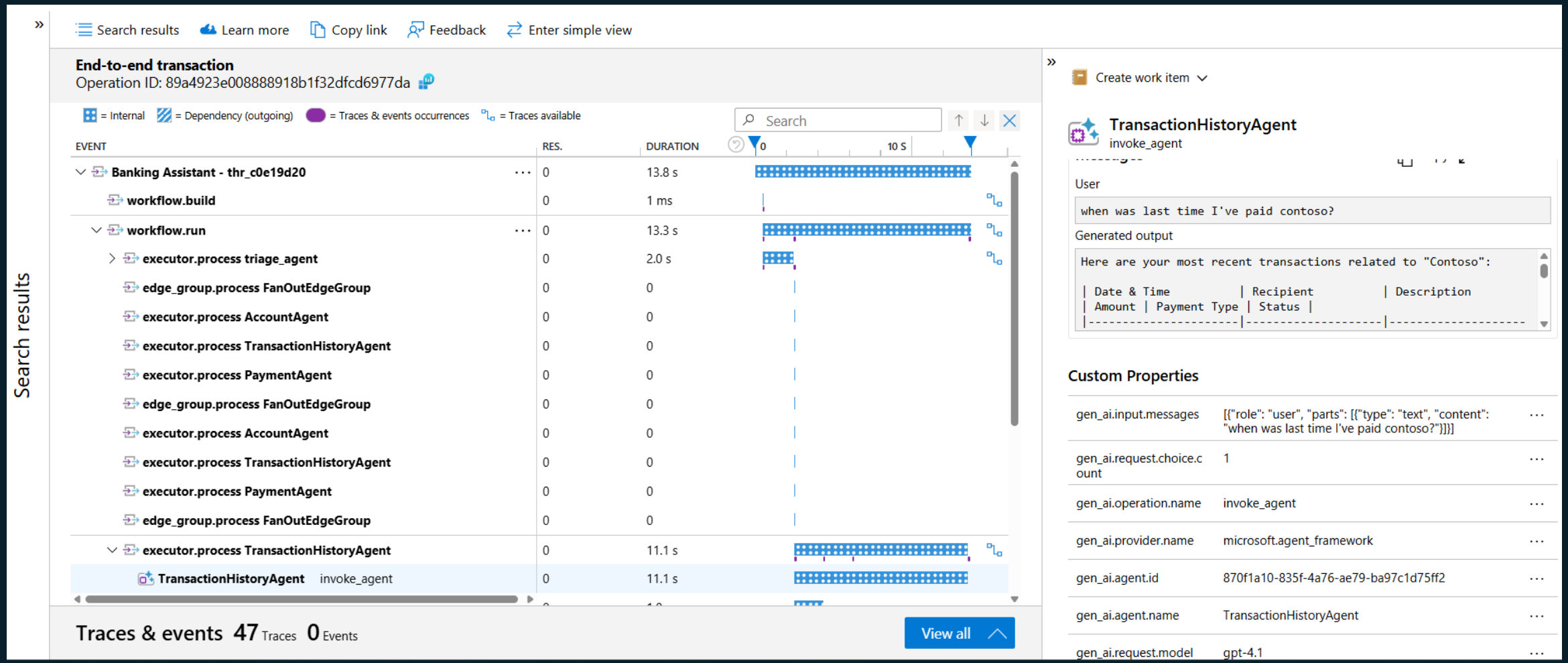
Map workflow events to chat protocol events:

[Microsoft Agent Framework Events]
[Chat Protocol Spec and Events]

```python
# Skip non-text events
if(event.type == "status" \
or event.type == "executor_invoked" ) \
or event.type == "executor_completed" \
or event.type == "superstep_started" \
or event.type == "superstep_completed" \
or event.type == "request_info":
    continue

if event.type == "handoff_sent":
   descriptive_title = f"Connected to {event.data.target} "
   handoff_result_task = CustomTask(title=descriptive_title, icon="check
   taskResultUpdate =  TaskItem(thread_id=thread_id,id=f"tsk_{uuid.uuid4
   yield ThreadItemAddedEvent(item=taskResultUpdate)
   continue

if event.type == "output":
    if isinstance(event.data, AgentResponseUpdate) \
    and event.executor_id != "triage_agent" \
    and event.data is not None and isinstance(event.data, AgentResponseU
    and event.data.contents \
    and isinstance(event.data.contents, list) \
    and all(item.type == "text" for item in event.data.contents):
    #In our case we just return text

        text_update = event.data.contents[0].text #type: ignore
        yield self._handle_text_content(thread_id=thread_id,message_id=m
```

[code]

Agent Tracing App Insights



Observability Setup

Stream agent traces to App Insights

```python
# Setup agent framework observability
if settings.AGENTS_TYPE == "foundry_v2" :
    configure_azure_monitor(
    connection_string=settings.APPLICATIONINSIGHTS_CONNECTION_STRING,
    resource=create_resource(),
    enable_live_metrics=True,
    )
else:
    configure_azure_monitor(
    connection_string=settings.APPLICATIONINSIGHTS_CONNECTION_STRING,
    resource=create_resource(),
    enable_live_metrics=True,
    )
    enable_instrumentation()
```

code

Sensitive data enabled on local dev

```json
{
    "name": "DEV - Chatkit Backend App",
    "type": "debugpy",
    "request": "launch",
    "module": "uvicorn",
    "args": [
        "app.main_chatkit_server:app",
        "--port", "8080"
    ],
    "env": {
        "PROFILE": "dev",
        "ENABLE_SENSITIVE_DATA": "true"
    },
```

code